

vim crash course

mohammed alshamsi

2021004826

mo.alshamsi@aurak.ac.ae

Coding Club at the American University of Ras Al Khaimah

October 26, 2025



outline

- 1 basics
 - normal mode
 - insert mode
 - movement
 - actions
 - visual mode
- 2 regex
- 3 command mode
 - :g and :v
 - :norm
- 4 macros
- 5 marks
- 6 concluding notes



- ▶ modal text editor.
- ▶ major modes: normal, insert, visual, visual line, visual block, command.
- ▶ there are others, like replace and insert-normal and terminal.
- ▶ major vim advantages: ergonomics, speed, lower mental overhead, lower boredom



normal mode

- ▶ in normal mode, the keys on your keyboard don't input numbers, letters, or symbols.
- ▶ instead, they perform functions: moving around the text, doing stuff with it, changing mode, or a combination.
- ▶ for example, `j` moves you down one line, and `dd` deletes the current line.



insert mode (briefly)

- ▶ press `i` in normal mode to enter insert mode
- ▶ works kind of like a normal text editor
- ▶ MOVEMENT IS SLOW IN INSERT MODE. DON'T STAY IN INSERT MODE.
- ▶ `<Esc>` to return to normal mode
- ▶ good to know at this point: `:w` to save file and `:q` to quit (normal mode)
- ▶ other ways of accessing insert mode: `I` `a` `A`
- ▶ (there's more, we'll get to them later)



movement

- ▶ lots of variety
- ▶ character-wise: `h j k l gj gk`
- ▶ word-wise: `w W b B e E ge gE`
- ▶ start/end of line: `_ $ 0`
- ▶ find character on line: `f⟨char⟩ F⟨char⟩ t⟨char⟩ T⟨char⟩ ; ,`
- ▶ screen-wise: `H L <C-f> <C-b> <C-d> <C-u>`
- ▶ whole file: `gg G ⟨line⟩G`
- ▶ search: `/⟨regex⟩ ?⟨regex⟩ n N`
- ▶ blocks, sections: `{ } [[ ]] [] ][`
- ▶ move to n'th column: `n|`
- ▶ actually, most of these can be preceded with a number to repeat the motion. e.g. `4j, 2n, 4fe, 32|`
- ▶ `:help motion.txt`



mindset: specify what action to do, then define the scope of the action with a motion

- ▶ cut: d⟨motion⟩ (example: dt.)
- ▶ copy (yank): y⟨motion⟩ (example: yj)
- ▶ change: c⟨motion⟩ (example: ct,)



other actions

many other actions don't have motions. some are shorthands for common d, y, or c tasks.

- ▶ current line: dd yy cc
- ▶ till end of line: D Y C
- ▶ paste: p P
- ▶ delete character: x
- ▶ alternatives for cl and cc: s S
- ▶ append next line to previous: J
- ▶ replace character: r⟨char⟩
- ▶ repeat last action: .
- ▶ indentation: >> <<
- ▶ zz zt zb
- ▶ <C-a> <C-x>
- ▶ entering insert mode: o O i I a A
- ▶ again, most can be preceded by a number.



more scopes

previous scopes for actions have all been motions you can do at any time in normal mode. some scopes don't count as motions.

- ▶ `i" i' i' iw iW ip is`
- ▶ `i{ i} i[ i] i( i) ib`
- ▶ all of the above but with a (e.g. `a"`, `aw`)



visual mode

lets you select text. you can then do something with that text, like cut with `d` or `x`.

- ▶ `v` for visual, `V` for visual line, `C-v` for visual block
- ▶ visual behaves like selection in normal text editors (holding shift or dragging mouse)
- ▶ visual line is line-wise
- ▶ visual block selects a square, and introduces some things: `g<C-a>` `g<C-x>`
- ▶ visual block is super good for editing multiple lines in parallel. use `A` and `I` to insert in left/right of block



regex (crash course in a crash course)

- ▶ please learn this properly in your own time. amazing tool.
- ▶ numbers and letters as normal
- ▶ grouping: `\(\\)`
- ▶ or operation: `\\|`
- ▶ consecutive occurrences: `\\{n\\}` `\\{n,\\}` `\\{,n\\}` `\\{m,n\\}` `*` `\\+`
- ▶ character class: `[abcdef]` `[^abcdef]` `[a-f]` `[a-f0-3]`
- ▶ some shorthands: `\\d` `\\u` `\\a` `\\l`
- ▶ lots of other stuff. `:help regex` for details



command mode

- ▶ access with :
- ▶ we saw this with :w and :q
- ▶ useful: :wq :qa :wa :wqa :q! :qa!
- ▶ search and replace: :s<regex>/<replacement>/<flags>
:%s<regex>/<replacement>/<flags>
- ▶ open the folder current file is in: :Ex
- ▶ open a file: :e <file>
- ▶ !<shellcommand>
- ▶ open a terminal window: :term
- ▶ very cool: q:
- ▶ :new, :vnew
- ▶ can open command mode from visual mode!



:g and :v

this is where things get really exciting

- ▶ run `<command>` on all lines matching `<regex>`: `:g<regex>/<command>`
- ▶ run `<command>` on all lines **not** matching `<regex>`: `:v<regex>/<command>`
- ▶ you can nest these ||
- ▶ run on all lines with `:%g` and `:%v`



:norm

run any sequence of non-command-mode motions and actions.

- ▶ <C-v> to access special characters like <Esc> and <C-w>
- ▶ can run :norm inside of :g and :v as well
- ▶ open command mode after selecting with visual line: : '<,'>norm
⟨operations⟩
- ▶ run on all lines with :%norm



registers

- ▶ access with "`<register>`"
- ▶ yank or cut into specific registers: `<register>x` `<register>d<motion>`
`<register>y<motion>`
- ▶ can paste from specific registers `<register>p<motion>` `<register>P<motion>`



this is where things get really exciting, part 2

- ▶ record sequence of actions into register: `q⟨register⟩`
- ▶ play back: `@⟨register⟩`
- ▶ play back most recently recorded: `Q`
- ▶ play back most recently played back: `@@`
- ▶ powerful pattern: record macro, select lines to repeat it on, then
`: '<, '>norm Q`



like registers but specifically for jumping around

- ▶ save location: `m<name>`
- ▶ jump to mark: `'<name>`<name>`
- ▶ good time to introduce these two gentlemen: `<C-o>` `<C-i>`
- ▶ some motions are 'jumps'. vim saves a history of positions you jumped to/from. `<C-o>` and `<C-i>` let you navigate that history.



other stuff

- ▶ vimtutor for a more relaxed exposition.
- ▶ :help <feature>
- ▶ i glossed over windows: <C-w>
- ▶ tabs are also cool: :tab
- ▶ <https://vimhelp.org>
- ▶ vim website <https://www.vim.org>
- ▶ nvim website <https://neovim.io>



thanks

i hope you learned something ☺

- ▶ join club pls thx
- ▶ use vim pls thx
- ▶ ???

